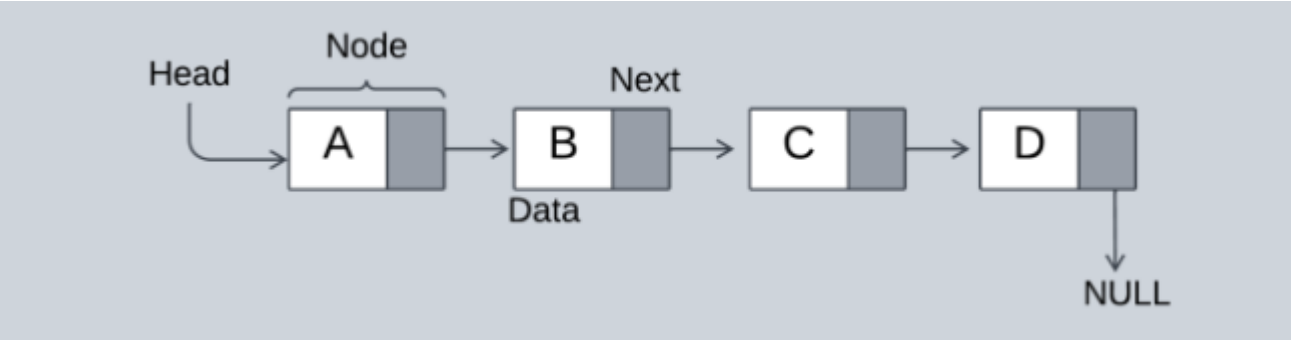
OBJECT ORIENTED PROGRAMMING USING C++

Study Materials

Linked list:

- Linked lists are less rigid in their storage structure and elements are usually not stored in contiguous locations, hence they need to be stored with additional tags giving a reference to the next element.



Types of Linked Lists:

Different Types of Linked Lists

a) Singly Linked List

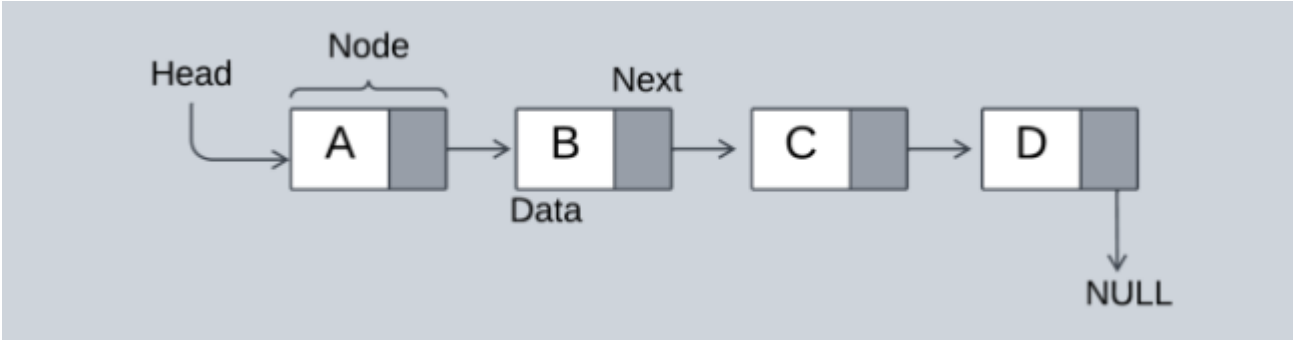
b) Doubly Linked List

c) Circular Linked List

d) Circular Doubly Linked List

Singly Linked List:

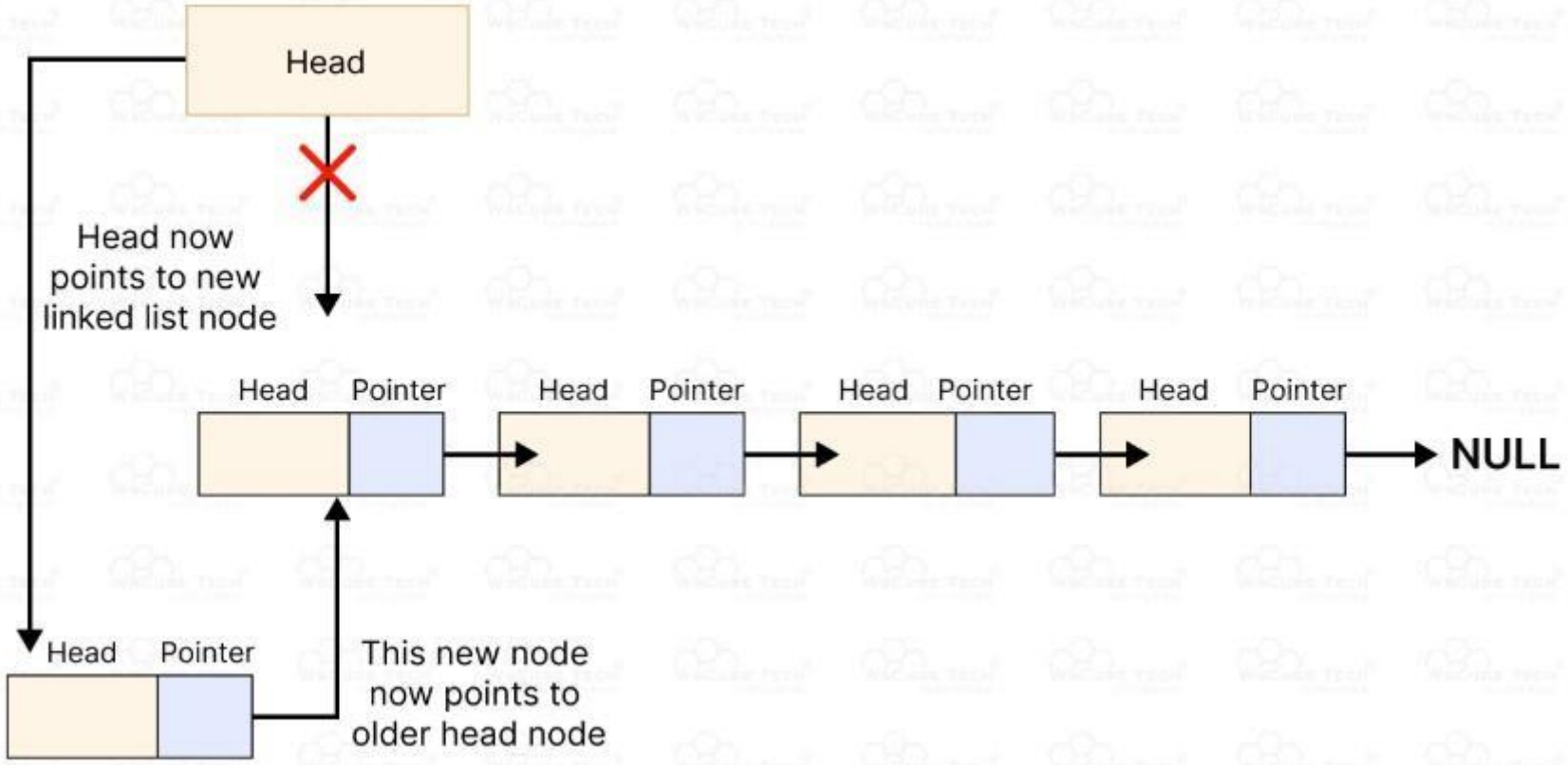
- A singly linked list in data structures is essentially a series of connected elements where each element, known as a node, contains a piece of data and a reference to the next node in the sequence.



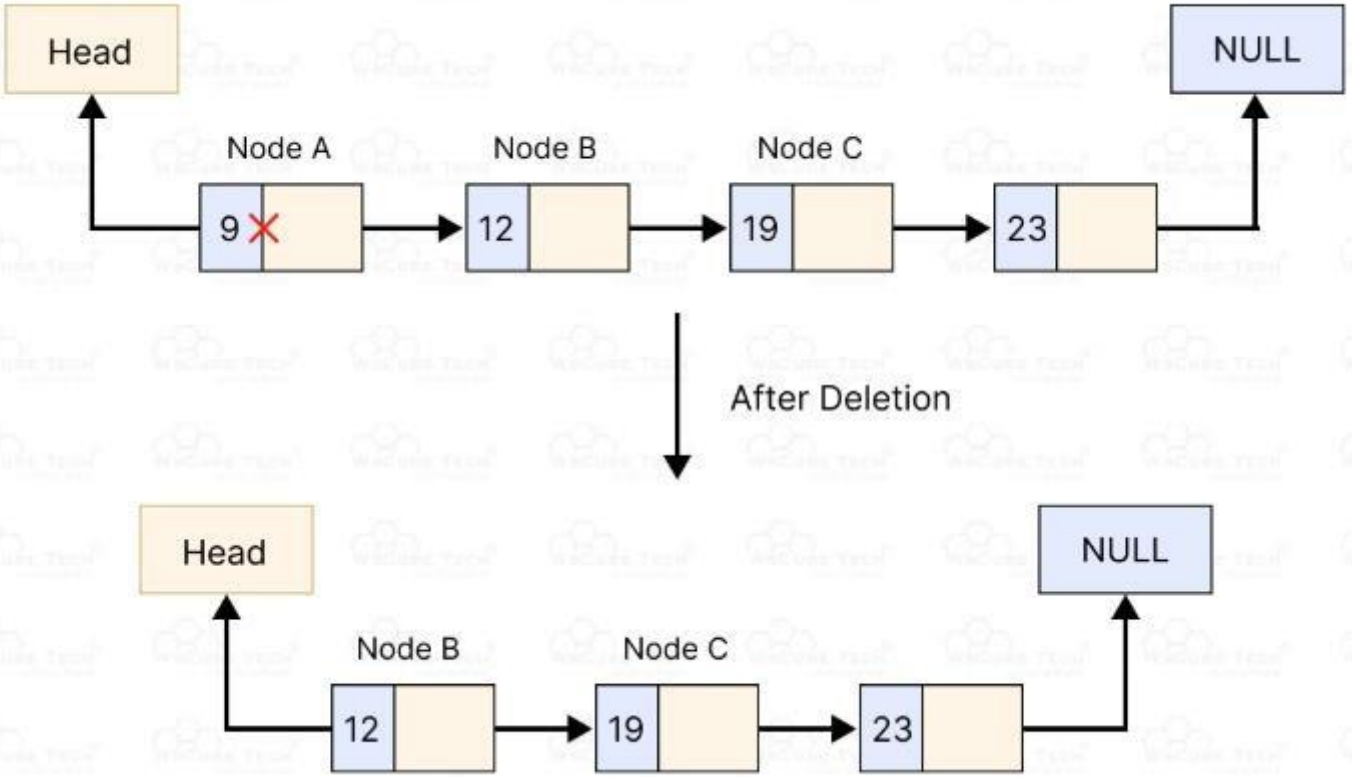
Operations:

- Traversal - access each element of the linked list
- Insertion - adds a new element to the linked list
- Deletion - removes the existing elements
- Search - find a node in the linked list
- Sort - sort the nodes of the linked list

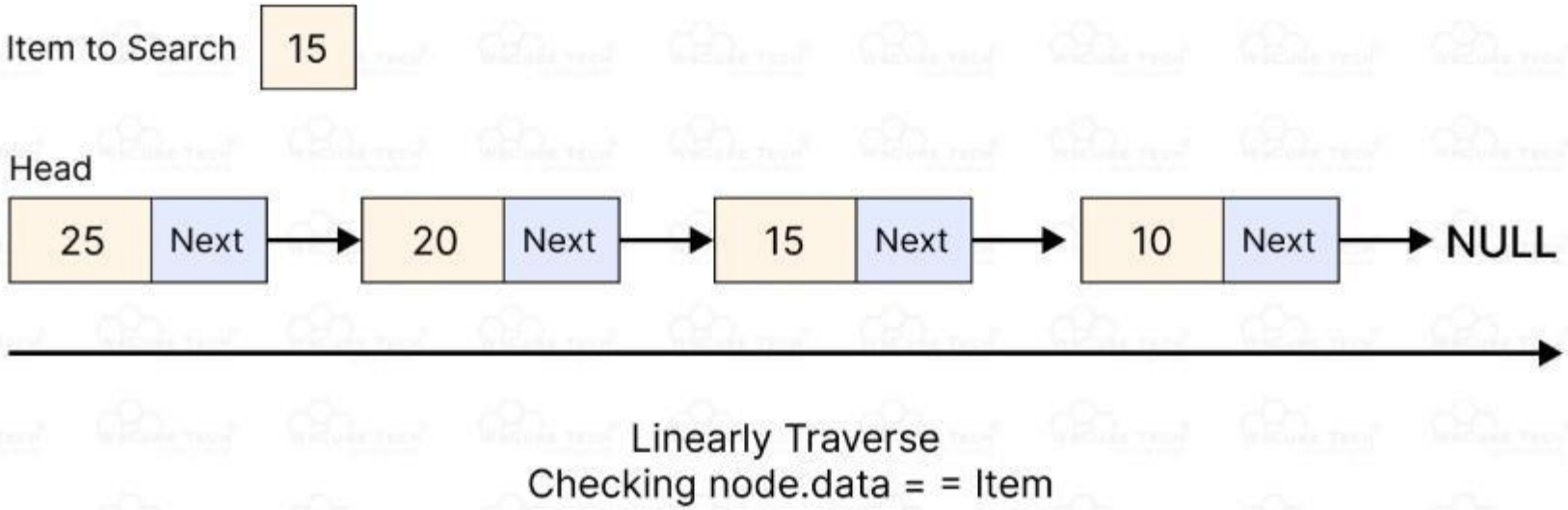
Insertion Operations:



Deletion Operations:



Search Operation:



Advantages:

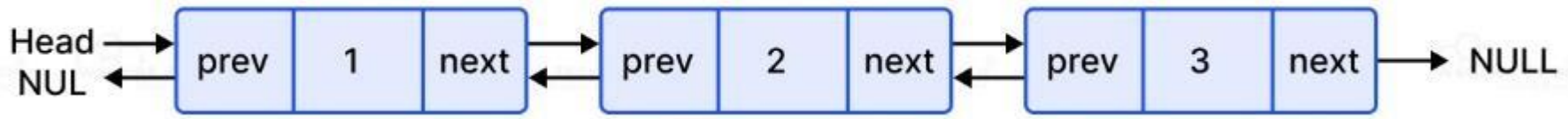
- Singly linked lists in data structures can grow and shrink during runtime as needed without a predefined size.
- They only allocate memory for nodes that are actually in use, reducing memory waste compared to pre-allocated data structures like arrays.
- Adding or removing nodes doesn't require shifting elements, which can be a costly operation in arrays.
- This is particularly beneficial at the beginning of the list.
- Unlike arrays, linked lists don’t reserve memory in advance, which can be more efficient for certain types of applications where the size of the data structure fluctuates.

Disadvantages:

- Accessing an element in a singly linked list requires traversal from the head to the point of interest, which can be time-consuming.
- Each node in a singly linked list requires additional memory for the pointer alongside the actual data.
- Implementing a singly linked list is more complex than using an array, particularly when it comes to handling pointers, which can introduce bugs like memory leaks.

Doubly Linked List:

- A doubly linked list is a type of data structure where each element, known as a node, holds data and two references: one pointing to the next node and one pointing to the previous node.
- This two-way linking allows for movement in both directions through the list, making it more versatile than a singly linked list which only allows movement in one direction.



Structure of a Doubly Linked List:

- In a data structure, a doubly linked list is represented using nodes that have three fields:
- Data
- A pointer to the next node (next)
- A pointer to the previous node (prev)

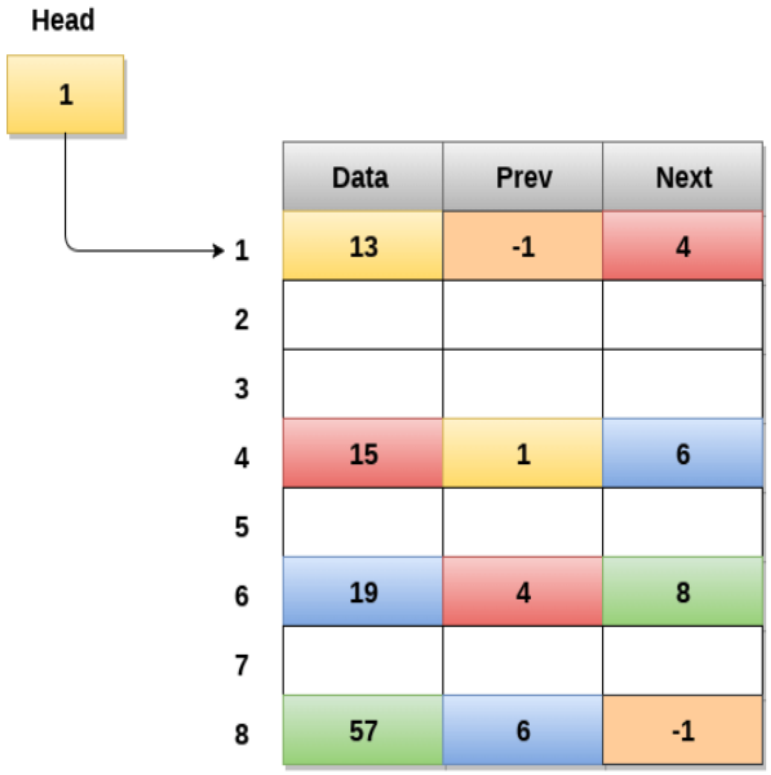
Working of a Doubly Linked List:

- A Doubly linked list node contains three fields:
- Left pointer,
- Information and
- Right pointer.
- The left pointer points to the node which is before the current node and the right pointer points to the node after the current node.
- A Doubly linked list allows backward traversing if required. All other functions are similar to linked list.
- It is different from the normal linked list by allowing the bidirectional traversal which in turn reduces the time complexity of any operation.

Characteristics of a doubly linked list:

- Each node contains two pointers: one pointing to the previous node and one pointing to the next node.
- The first node’s previous pointer points to null and the last node’s next pointer points to null.
- The linked list can be traversed in both forward and backward directions.
- The size of the linked list can be dynamically adjusted based on the number of elements added or removed.
- Insertions and deletions can be performed in constant time at both the beginning and the end of the list.
- Access and search operations have O(n) time complexity, where n is the number of elements in the list.
- It requires more memory compared to a singly linked list, as each node has two pointers.

Memory Representation of a doubly linked list



Operations:

SN	Operation	Description
1	Insertion at beginning	Adding the node into the linked list at beginning.
2	Insertion at end	Adding the node into the linked list to the end.
3	Insertion after specified node	Adding the node into the linked list after the specified node.
4	Deletion at beginning	Removing the node from beginning of the list
5	Deletion at the end	Removing the node from end of the list.
6	Deletion of the node having given data	Removing the node which is present just after the node containing the given data.
7	Searching	Comparing each node data with the item to be searched and return the location of the item in the list if the item found else return null.
8	Traversing	Visiting each node of the list at least once in order to perform some specific operation like searching, sorting, display, etc.

Applications:

- Doubly linked list can be used in navigation systems where both forward and backward traversal is required.
- It can be used to implement different tree data structures.
- It can be used to implement undo/redo operations.
- Doubly linked lists are used in web page navigation in both forward and backward directions.
- It can be used in games like a deck of cards.

Advantages:

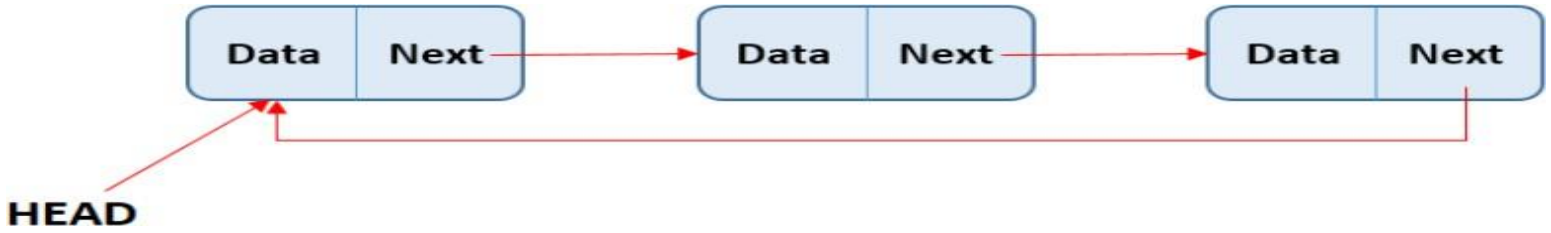
- Bidirectional Navigation: Allows traversal in both forward and backward directions, facilitating easier and more flexible data manipulation.
- Easier Insertion and Deletion: Nodes can be added or removed from both ends and the middle of the list without needing to traverse the entire list, especially if the tail pointer is maintained.
- Efficient Operations at Both Ends: Adding or removing elements at the beginning and the end is efficient because both head and tail pointers provide direct access to the endpoints of the list.
- Dynamic Size: The size of the list can increase or decrease dynamically, which is efficient for memory usage since it allocates space only as needed.

Disadvantages:

- Increased Memory Usage: Each node requires extra memory for an additional pointer (previous pointer), which can be significant in memory-constrained environments.
- Complexity: Managing two pointers (next and prev) per node increases the complexity of the operations, making the code more prone to errors such as memory leaks and pointer corruption.
- Slower Individual Operations: The overhead of maintaining an extra pointer can slightly slow down operations compared to singly linked lists, as more pointer operations are required.
- Overhead in Memory Management.

Circular Linked List:

- A circular linked list is a special type of linked list where all the nodes are connected to form a circle.
- Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node.
- This means that you can keep traversing the list without ever reaching a NULL value.



Types of Circular Linked Lists:

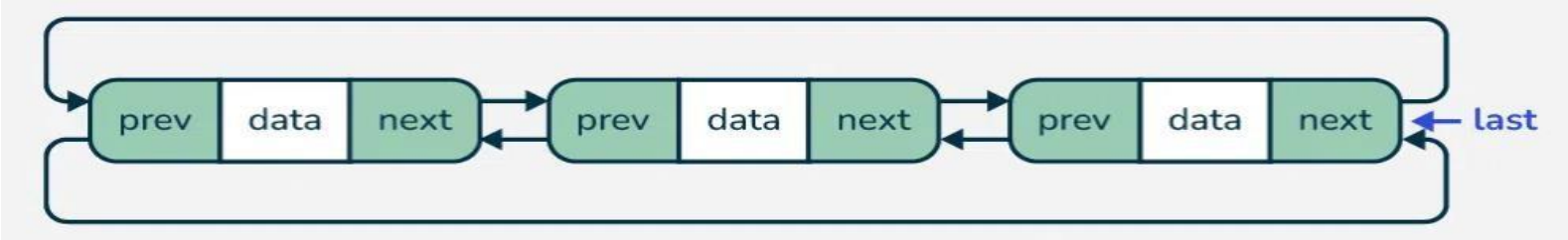
- We can create a circular linked list from both singly linked lists and doubly linked lists.
- So, circular linked list are basically of two types:
- Singly linked lists
- Doubly linked lists.

Circular Singly Linked List:

- In Circular Singly Linked List, each node has just one pointer called the “next” pointer.
- The next pointer of last node points back to the first node and this results in forming a circle.
- In this type of Linked list we can only move through the list in one direction.

Circular Doubly Linked List:

- In circular doubly linked list, each node has two pointers prev and next, similar to doubly linked list.
- The prev pointer points to the previous node and the next points to the next node.
- Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.



Operations on the Circular Linked list:

1. Insertion:

- Insertion at the empty list
- Insertion at the beginning
- Insertion at the end
- Insertion at the given position

Operations on the Circular Linked list:

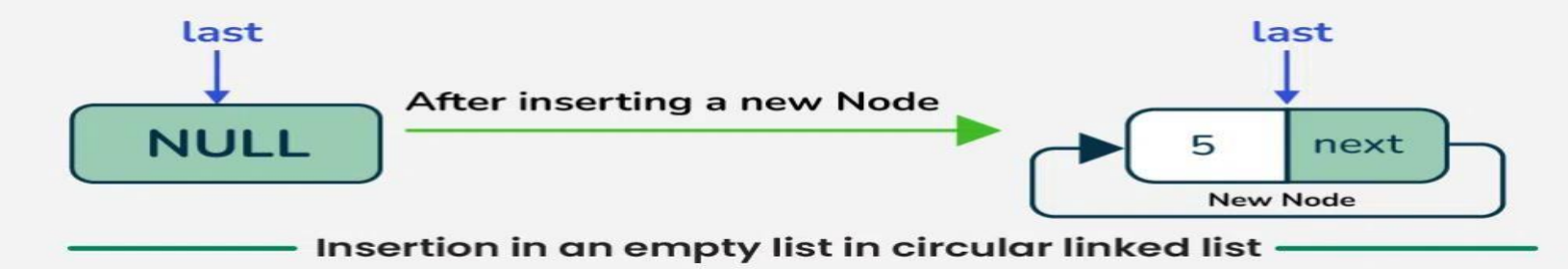
2. Deletion:

- Delete the first node
- Delete the last node
- Delete the node from any position

3. Searching:

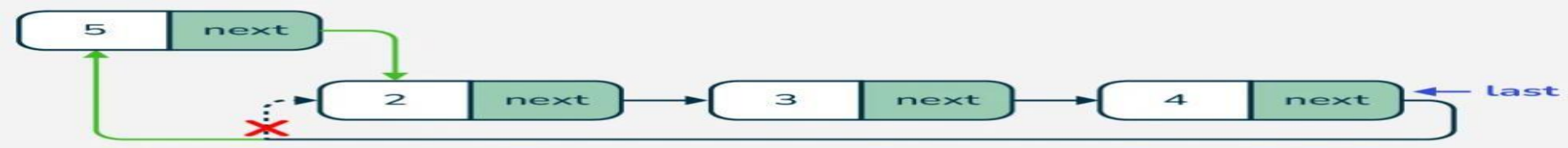
Insertion in an empty List:

- To insert a node in empty circular linked list, creates a new node with the given data, sets its next pointer to point to itself, and updates the last pointer to reference this new node.



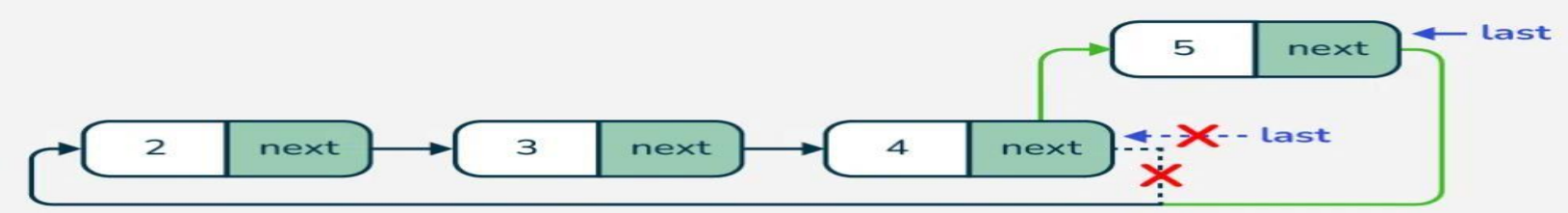
Insertion at specific position :

- To insert a new node at the beginning of a circular linked list, we first create the new node and allocate memory for it. If the list is empty (indicated by the last pointer being NULL), we make the new node point to itself.
- If the list already contains nodes then we set the new node's next pointer to point to the current head of the list (which is last->next), and then update the last node's next pointer to point to the new node. This maintains the circular structure of the list.



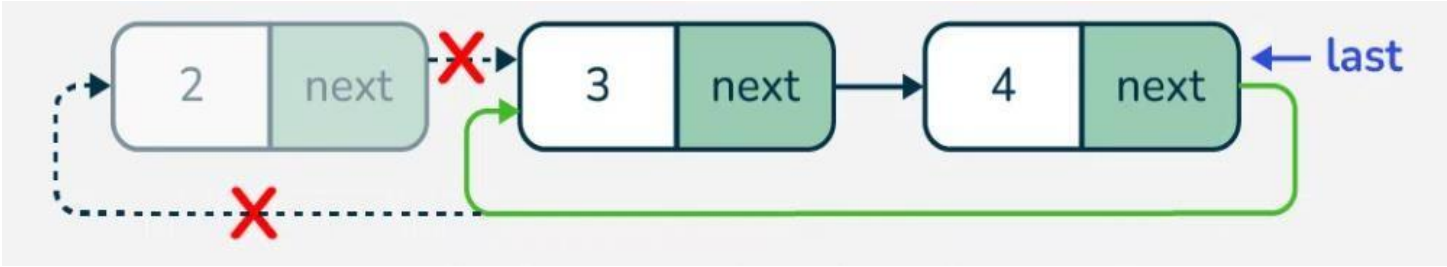
Insertion at the end:

- To insert a new node at the end of a circular linked list, we first create the new node and allocate memory for it.
- If the list is empty (mean, last or tail pointer being NULL), we initialize the list with the new node and making it point to itself to form a circular structure.
- If the list already contains nodes then we set the new node's next pointer to point to the current head (which is tail->next), then update the current tail's next pointer to point to the new node.



Delete the first node:

- To delete the first node of a circular linked list, we first check if the list is empty. If it is then we print a message and return NULL.
- If the list contains only one node (the head is the same as the last) then we delete that node and set the last pointer to NULL.
- If there are multiple nodes then we update the last->next pointer to skip the head node and effectively removing it from the list. We then delete the head node to free the allocated memory. Finally, we return the updated
- last pointer, which still points to the last node in the list.

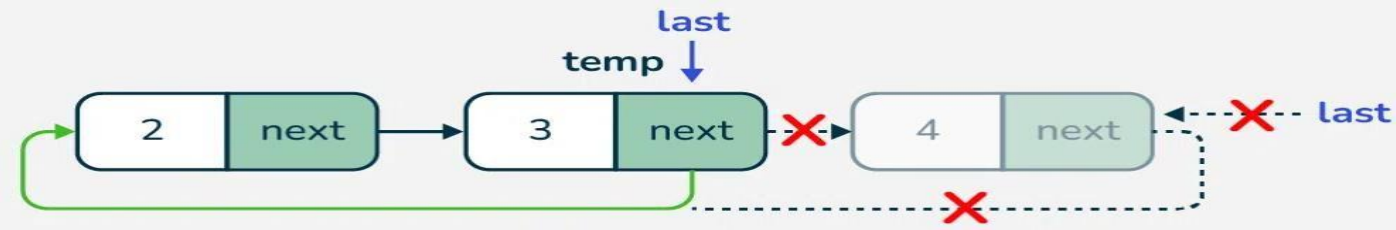


Delete a specific node:

- To delete a specific node from a circular linked list, we first check if the list is empty. If it is then we print a message and return nullptr.
- If the list contains only one node and it matches the key then we delete that node and set last to nullptr. If the node to be deleted is the first node then we update the next pointer of the last node to skip the head node and delete the head.

Deletion at the end:

- To delete the last node in a circular linked list, we first check if the list is empty. If it is, we print a message and return nullptr.
- If the list contains only one node (where the head is the same as the last), we delete that node and set last to nullptr.
- For lists with multiple nodes, we need to traverse the list to find the second last node. We do this by starting from the head and moving through the list until we reach the node whose next pointer points to last.



Applications:

- It is used for time-sharing among different users, typically through a Round-Robin scheduling mechanism.
- In multiplayer games, a circular linked list can be used to switch between players. After the last player's turn, the list cycles back to the first player.
- Circular linked lists are often used in buffering applications, such as streaming data, where data is continuously produced and consumed.
- In media players, circular linked lists can manage playlists, this allowing users to loop through songs continuously.
- Browsers use circular linked lists to manage the cache.
- This allows you to navigate back through your browsing history efficiently by pressing the BACK button.